Step 5 — The Full Team: a Supervisor Routing 4 Specialists via Command — Hints

Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — Who decides, and how the state is shared
- Hint 2 — The node wrappers and the graph

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — Who decides, and how the state is shared

Up to now `main.py` decided the order. Now a **Supervisor** decides, from the shared state:

- no research yet → Research
- research found nothing → stop
- no findings → Analysis
- accepted, or at the revision limit → stop
- no draft → Writer
- otherwise → Reviewer

Every specialist hands back to the Supervisor when it's done. The Reviewer sends work back to the Writer simply by **clearing the draft** (Doc11).

`state.py` — `TeamState` with **one writer per field** (Doc11): `research_notes` + `research_done` (Research), `findings` (Analysis), `draft` (Writer), `feedback_history` + `accepted` + `revision_count` (Reviewer), `routing_log` (Supervisor only). `research_done` is set even when nothing was found, so Research never runs twice.

`agents/supervisor.py`:

```
MAX_REVISIONS = 3

def decide_next(state: TeamState) -> tuple[str, str]:
    ...  # (next step, reason) — the rules above, in order

def supervisor(state: TeamState) -> Command:
    ...  # log {"step", "reason"}, then Command(goto=...)
```

Test `decide_next()` on hand-made states **before** building the graph.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The node wrappers and the graph

Each agent file gets a thin wrapper; the `run_...()` function inside stays exactly the same:

```
research_node(state):
    notes = run_research(state["topic"])
    return Command(update={"research_notes": notes,
                           "research_done": True},
                   goto="supervisor")
```

`graph.py`: add the 5 nodes and **one** edge, `START → "supervisor"`. Every other move is a `Command(goto=...)`.

`reviewer_node` holds Step 2's loop logic:

```
verdict = run_reviewer(findings, draft)
pass -> {"accepted": True}
fail -> revision_count + 1, feedback_history + [feedback],
        and if revision_count < MAX_REVISIONS: draft = ""
```

`writer_node` passes the newest feedback (`feedback_history[-1]`, if any) to `run_writer()`.

`main.py` runs `graph.invoke(start_state(topic))` and passes the result to the **same** `print_report(result)` from Step 2 — the graph's state has the same keys. `print_report()` only gains two things: the routing log at the top, and a "nothing is known" branch (`elif result["research_notes"] == ""`). `write_with_review()` is deleted: the Supervisor and `reviewer_node` now do its job.

Tests: `test_supervisor.py` walks through every rule with hand-made states; the integration test runs the weak-draft check, a full pipeline and an unknown topic.

Full solution: Show me the solution